

MSCFEE 620 Derivative Pricing — GWP Q5-Q18

This notebook provides the computational answers for **Questions 5-18** of Group Work Project #1.

Scope:

- Q5-Q7: European call and put pricing, Delta, and volatility sensitivity using a binomial tree.
- Q8-Q10: American call and put pricing, Delta, and volatility sensitivity using a binomial tree.
- Q11-Q14: Put-call parity checks and European-versus-American comparisons.
- Q15-Q16: European call and put pricing using a trinomial tree across five strikes.
- Q17-Q18: American call and put pricing using a trinomial tree across the same five strikes.

Unless stated otherwise, we use the Step 1 parameters:

[$S_0=100$, $r=5\%$, $\sigma=20\%$, $T=3$ months $=0.25$.]

The stock is assumed to pay no dividends. All option prices in answer tables are reported in dollars and rounded to the nearest cent.

Setup

We use recombining binomial and trinomial trees. The implementation supports both European exercise and American early exercise, so the same pricing functions can be reused across Q5-Q18.

```
In [1]: import math
import numpy as np
import pandas as pd
from IPython.display import display

pd.set_option("display.float_format", lambda x: f"{x:0.6f}")

S0 = 100.0
r = 0.05
sigma = 0.20
T = 0.25
K_ATM = 100.0

BINOMIAL_STEPS = 1000
TRINOMIAL_STEPS = 1000
```

Pricing methodology

Binomial tree

For a tree with (N) steps, we set

$$[\Delta t = \frac{T}{N}, \quad u = e^{\sigma \sqrt{\Delta t}}, \quad d = \frac{1}{u}.]$$

The risk-neutral probability is

$$[p = \frac{e^{r\Delta t} - d}{u - d}.]$$

At maturity, the option value is its payoff:

$$[C_T = \max(S_T - K, 0), \quad P_T = \max(K - S_T, 0).]$$

For a European option, we move backward through the tree using

$$[V_t = e^{-r\Delta t} \left[pV_u + (1-p)V_d \right].]$$

For an American option, we compare continuation value with immediate exercise value at every node:

$$[V_t = \max(\text{intrinsic value}, e^{-r\Delta t} \left[pV_u + (1-p)V_d \right]).]$$

Trinomial tree

The trinomial tree allows the stock to move up, stay in the middle, or move down over each step. In this notebook, we use a recombining tree with

$$[u = e^{\sigma \sqrt{2\Delta t}},]$$

and risk-neutral probabilities selected so that the one-step drift and variance are consistent with the model inputs. The backward recursion is

$$[V_t = e^{-r\Delta t} \left[p_u V_u + p_m V_m + p_d V_d \right],]$$

with the same American early-exercise maximum when `american=True`.

```
In [2]: def payoff(stock, strike, option_type):
    stock = np.asarray(stock, dtype=float)
    if option_type == "call":
        return np.maximum(stock - strike, 0.0)
    if option_type == "put":
        return np.maximum(strike - stock, 0.0)
    raise ValueError("option_type must be 'call' or 'put'")

def binomial_tree_price(S0, K, r, sigma, T, steps, option_type, american=
    """CRR binomial option price and time-0 delta."""
    dt = T / steps
    u = math.exp(sigma * math.sqrt(dt))
    d = 1.0 / u
    disc = math.exp(-r * dt)
    p = (math.exp(r * dt) - d) / (u - d)

    if not 0.0 <= p <= 1.0:
```

```

        raise ValueError(f"Risk-neutral probability outside [0, 1]: {p}")

    j = np.arange(steps + 1)
    stock = S0 * (u ** j) * (d ** (steps - j))
    values = payoff(stock, K, option_type)
    first_step_values = None

    for step in range(steps - 1, -1, -1):
        values = disc * (p * values[1:] + (1.0 - p) * values[:-1])

        if american:
            j = np.arange(step + 1)
            stock = S0 * (u ** j) * (d ** (step - j))
            values = np.maximum(values, payoff(stock, K, option_type))

        if step == 1:
            first_step_values = values.copy()

    price = float(values[0])
    delta = None
    if first_step_values is not None:
        s_down = S0 * d
        s_up = S0 * u
        delta = float((first_step_values[1] - first_step_values[0]) / (s_

    return {
        "price": price,
        "delta": delta,
        "steps": steps,
        "u": u,
        "d": d,
        "p": p,
    }

def trinomial_tree_price(S0, K, r, sigma, T, steps, option_type, american):
    """Recombining trinomial tree option price."""
    dt = T / steps
    disc = math.exp(-r * dt)
    u = math.exp(sigma * math.sqrt(2.0 * dt))

    a = math.exp(r * dt / 2.0)
    b = math.exp(sigma * math.sqrt(dt / 2.0))
    c = 1.0 / b
    pu = ((a - c) / (b - c)) ** 2
    pdn = ((b - a) / (b - c)) ** 2
    pm = 1.0 - pu - pdn

    if min(pu, pm, pdn) < -1e-12:
        raise ValueError(f"Invalid trinomial probabilities: pu={pu}, pm={

    j = np.arange(-steps, steps + 1)
    stock = S0 * (u ** j)
    values = payoff(stock, K, option_type)

    for step in range(steps - 1, -1, -1):
        values = disc * (pdn * values[:-2] + pm * values[1:-1] + pu * val

        if american:
            j = np.arange(-step, step + 1)

```

```

        stock = S0 * (u ** j)
        values = np.maximum(values, payoff(stock, K, option_type))

    return {
        "price": float(values[0]),
        "steps": steps,
        "pu": pu,
        "pm": pm,
        "pd": pdn,
    }

def cents(x):
    return round(float(x), 2)

def four_decimals(x):
    return round(float(x), 4)

```

Q5 — European call and put using a binomial tree

We first check convergence across several step counts. The final reported price uses **1000** steps because the prices are stable to the nearest cent by that point.

```

In [3]: convergence_rows = []
for n in [25, 50, 100, 250, 500, 1000]:
    call = binomial_tree_price(S0, K_ATM, r, sigma, T, n, "call", american=
    put = binomial_tree_price(S0, K_ATM, r, sigma, T, n, "put", american=
    convergence_rows.append({
        "steps": n,
        "European call": cents(call["price"]),
        "European put": cents(put["price"]),
    })

q5_convergence = pd.DataFrame(convergence_rows)
display(q5_convergence)

euro_call = binomial_tree_price(S0, K_ATM, r, sigma, T, BINOMIAL_STEPS, "
euro_put = binomial_tree_price(S0, K_ATM, r, sigma, T, BINOMIAL_STEPS, "p

q5_answer = pd.DataFrame([
    {"question": "Q5", "style": "European", "option": "call", "strike": K
    {"question": "Q5", "style": "European", "option": "put", "strike": K_
])

display(q5_answer)

```

	steps	European call	European put
0	25	4.650000	3.410000
1	50	4.600000	3.350000
2	100	4.610000	3.360000
3	250	4.610000	3.370000
4	500	4.610000	3.370000
5	1000	4.610000	3.370000

	question	style	option	strike	steps	price
0	Q5	European	call	100.000000	1000	4.610000
1	Q5	European	put	100.000000	1000	3.370000

Q5 answer. With a 1000-step binomial tree, the ATM European call is about **4.61** and the ATM European put is about **3.37**.

The convergence table shows that increasing the tree from 250 to 500 to 1000 steps changes the estimated prices only slightly. Therefore, 1000 steps provides a stable and reproducible estimate for this exercise.

Q6 — Delta for the European call and put

The binomial-tree delta is computed from the first up/down node values:

$$[\Delta_0 = \frac{V_u - V_d}{S_u - S_d}.]$$

```
In [4]: q6_answer = pd.DataFrame([
    {"question": "Q6", "style": "European", "option": "call", "delta": fo
    {"question": "Q6", "style": "European", "option": "put", "delta": fou
])
display(q6_answer)
```

	question	style	option	delta
0	Q6	European	call	0.569400
1	Q6	European	put	-0.430600

Q6 answer. The European call Delta is positive because the call value increases when the underlying stock price increases. The European put Delta is negative because the put value decreases when the underlying stock price increases.

The call Delta is about **0.5694**, while the put Delta is about **-0.4306**. The difference between these Deltas is close to 1, which is consistent with put-call parity for non-dividend-paying European options.

Q7 — European price sensitivity to a volatility increase

We raise volatility from 20% to 25% and re-price the European call and put. The table reports both the total price change for the 5-volatility-point increase and the approximate price change per 1-volatility-point increase.

```
In [5]: euro_call_high_vol = binomial_tree_price(S0, K_ATM, r, 0.25, T, BINOMIAL_S
euro_put_high_vol = binomial_tree_price(S0, K_ATM, r, 0.25, T, BINOMIAL_S

q7_answer = pd.DataFrame([
    {
        "question": "Q7",
        "style": "European",
        "option": "call",
        "price at 20% vol": cents(euro_call["price"]),
        "price at 25% vol": cents(euro_call_high_vol["price"]),
        "price change for +5 vol points": cents(euro_call_high_vol["price"]
        "approx change per +1 vol point": cents((euro_call_high_vol["pric
    },
    {
        "question": "Q7",
        "style": "European",
        "option": "put",
        "price at 20% vol": cents(euro_put["price"]),
        "price at 25% vol": cents(euro_put_high_vol["price"]),
        "price change for +5 vol points": cents(euro_put_high_vol["price"]
        "approx change per +1 vol point": cents((euro_put_high_vol["price
    },
])
display(q7_answer)
```

	question	style	option	price at 20% vol	price at 25% vol	price change for +5 vol points	approx change per +1 vol point
0	Q7	European	call	4.610000	5.600000	0.980000	0.200000
1	Q7	European	put	3.370000	4.350000	0.980000	0.200000

Q7 answer. Both European options become more valuable when volatility rises. Higher volatility widens the distribution of possible terminal stock prices, which increases the value of optionality.

For these ATM European options, the call and put price changes are almost identical. This is consistent with put-call parity: when (S_0), (K), (r), and (T) are fixed, a volatility change affects both sides in a similar way.

Q8 — American call and put using a binomial tree

We now repeat Q5 for American-style options.

```
In [6]: amer_call = binomial_tree_price(S0, K_ATM, r, sigma, T, BINOMIAL_STEPS, "
amer_put = binomial_tree_price(S0, K_ATM, r, sigma, T, BINOMIAL_STEPS, "p

q8_answer = pd.DataFrame([
    {"question": "Q8", "style": "American", "option": "call", "strike": K
    {"question": "Q8", "style": "American", "option": "put", "strike": K_
])
display(q8_answer)
```

	question	style	option	strike	steps	price
0	Q8	American	call	100.000000	1000	4.610000
1	Q8	American	put	100.000000	1000	3.480000

Q8 answer. The American call is about **4.61**, which matches the European call in this no-dividend setting. The American put is about **3.48**, which is higher than the European put because early exercise can add value for puts.

Q9 — Delta for the American call and put

We compute the same first-step binomial delta for the American options.

```
In [7]: q9_answer = pd.DataFrame([
    {"question": "Q9", "style": "American", "option": "call", "delta": fo
    {"question": "Q9", "style": "American", "option": "put", "delta": fou
])
display(q9_answer)
```

	question	style	option	delta
0	Q9	American	call	0.569400
1	Q9	American	put	-0.449500

Q9 answer. The American call Delta is positive and matches the European call Delta in this no-dividend setting. The American put Delta is negative and slightly more negative than the European put Delta, reflecting the additional early-exercise feature.

Q10 — American price sensitivity to a volatility increase

We repeat the volatility-sensitivity calculation for American-style options, again increasing volatility from 20% to 25%.

```
In [8]: amer_call_high_vol = binomial_tree_price(S0, K_ATM, r, 0.25, T, BINOMIAL_
amer_put_high_vol = binomial_tree_price(S0, K_ATM, r, 0.25, T, BINOMIAL_S

q10_answer = pd.DataFrame([
    {
        "question": "Q10",
        "style": "American",
```

```

        "option": "call",
        "price at 20% vol": cents(amer_call["price"]),
        "price at 25% vol": cents(amer_call_high_vol["price"]),
        "price change for +5 vol points": cents(amer_call_high_vol["price"] - amer_call["price"]),
        "approx change per +1 vol point": cents((amer_call_high_vol["price"] - amer_call["price"]) / 5),
    },
    {
        "question": "Q10",
        "style": "American",
        "option": "put",
        "price at 20% vol": cents(amer_put["price"]),
        "price at 25% vol": cents(amer_put_high_vol["price"]),
        "price change for +5 vol points": cents(amer_put_high_vol["price"] - amer_put["price"]),
        "approx change per +1 vol point": cents((amer_put_high_vol["price"] - amer_put["price"]) / 5),
    },
])
display(q10_answer)

```

	question	style	option	price at 20% vol	price at 25% vol	price change for +5 vol points	approx change per +1 vol point
0	Q10	American	call	4.610000	5.600000	0.980000	0.200000
1	Q10	American	put	3.480000	4.460000	0.980000	0.200000

Q10 answer. The American call price change matches the European call price change because early exercise is not optimal for a non-dividend-paying call. The American put also increases in value when volatility rises, although the early-exercise feature slightly changes the sensitivity relative to the European put.

Q11 — European put-call parity confirmation

For non-dividend-paying European options, put-call parity is:

$$[C + K e^{-rT}] = P + S_0.$$

We verify the equality using the Q5 European binomial prices.

```

In [9]: pv_strike = K_ATM * math.exp(-r * T)

q11_left = euro_call["price"] + pv_strike
q11_right = euro_put["price"] + S0
q11_difference = q11_left - q11_right

q11_answer = pd.DataFrame([
    {
        "question": "Q11",
        "style": "European",
        "call price": cents(euro_call["price"]),
        "put price": cents(euro_put["price"]),
        "C + K exp(-rT)": round(q11_left, 6),
        "P + S0": round(q11_right, 6),
        "difference": round(q11_difference, 8),
        "holds within 1 cent": abs(q11_difference) <= 0.01,
    }
])
display(q11_answer)

```



```

    }
  })

display(q11_answer)

```

	question	style	call price	put price	C + K exp(-rT)	P + S0	difference	hold within 1 cen
0	Q11	European	4.610000	3.370000	103.371779	103.371779	-0.000000	True

Q11 answer. The European call and put prices satisfy put-call parity within rounding. This is expected because European options have a single exercise date, so the present value of the strike is unambiguous.

Q12 — American put-call parity check and bounds

The strict European parity equality does not generally apply to American options because the holder may choose early exercise. For non-dividend-paying American options, the relevant no-arbitrage bounds are:

$$[S_0 - K \leq C_A - P_A \leq S_0 - K e^{-rT}].$$

We therefore report two checks: first, the European-style equality check; second, the American no-arbitrage bounds.

```

In [10]: q12_left = amer_call["price"] + pv_strike
q12_right = amer_put["price"] + S0
american_spread = amer_call["price"] - amer_put["price"]
lower_bound = S0 - K_ATM
upper_bound = S0 - pv_strike

q12_equality = pd.DataFrame([
    {
        "question": "Q12",
        "check": "European-style equality",
        "C_A + K exp(-rT)": round(q12_left, 6),
        "P_A + S0": round(q12_right, 6),
        "difference": round(q12_left - q12_right, 6),
        "equality holds within 1 cent": abs(q12_left - q12_right) <= 0.01
    }
])

q12_bounds = pd.DataFrame([
    {
        "question": "Q12",
        "check": "American no-arbitrage bounds",
        "lower bound S0 - K": round(lower_bound, 6),
        "C_A - P_A": round(american_spread, 6),
        "upper bound S0 - K exp(-rT)": round(upper_bound, 6),
        "bounds hold": lower_bound - 1e-10 <= american_spread <= upper_bound
    }
])

```

```
display(q12_equality)
display(q12_bounds)
```

	question	check	$C_A + K \exp(-rT)$	$P_A + S_0$	difference	equality holds within 1 cent
0	Q12	European-style equality	103.371779	103.479342	-0.107563	False

	question	check	lower bound $S_0 - K$	$C_A - P_A$	upper bound $S_0 - K \exp(-rT)$	bounds hold
0	Q12	American no-arbitrage bounds	0.000000	1.134657	1.242220	True

Q12 answer. The European-style equality does not hold for the American put and call in this example. However, the American no-arbitrage bounds do hold, which is the correct relationship for American options.

Q13 — European call versus American call

An American option includes all European exercise rights plus the possibility of earlier exercise. Therefore:

$$[C_E \leq C_A.]$$

```
In [11]: q13_answer = pd.DataFrame([
    {
        "question": "Q13",
        "option": "call",
        "European price": cents(euro_call["price"]),
        "American price": cents(amer_call["price"]),
        "American - European": cents(amer_call["price"] - euro_call["price"]),
        "C_E <= C_A": euro_call["price"] <= amer_call["price"] + 1e-10,
    }
])

display(q13_answer)
```

	question	option	European price	American price	American - European	$C_E \leq C_A$
0	Q13	call	4.610000	4.610000	0.000000	True

Q13 answer. The European call is less than or equal to the American call. In this example, the two values are equal to cents because the underlying stock pays no dividends, so early exercise of the American call is not optimal. More generally, an American option cannot be worth less than the corresponding European option because it includes all European exercise rights plus the right to exercise earlier.

Q14 — European put versus American put

For puts, early exercise can be valuable. We therefore expect:

$P_E \leq P_A$

```
In [12]: q14_answer = pd.DataFrame([
    {
        "question": "Q14",
        "option": "put",
        "European price": cents(euro_put["price"]),
        "American price": cents(amer_put["price"]),
        "American - European": cents(amer_put["price"] - euro_put["price"])
        "P_E <= P_A": euro_put["price"] <= amer_put["price"] + 1e-10,
    }
])

display(q14_answer)
```

	question	option	European price	American price	American - European	P_E <= P_A
0	Q14	put	3.370000	3.480000	0.110000	True

Q14 answer. The American put is more expensive than the European put in this example. The difference reflects the value of the early-exercise right. The inequality ($P_E \leq P_A$) should hold for otherwise identical options because the American holder has at least the same rights as the European holder.

Q15-Q18 setup — five strikes for trinomial trees

Moneyness is measured as K / S_0 . We use five strikes around the current stock price:

- $K = 90$: call deep ITM / put deep OTM
- $K = 95$: call ITM / put OTM
- $K = 100$: ATM
- $K = 105$: call OTM / put ITM
- $K = 110$: call deep OTM / put deep ITM

The trinomial calculations also use **1000** steps, matching the binomial section and providing stable estimates to the nearest cent.

```
In [13]: strike_rows = [
    {"strike": 90.0, "K/S0": 0.90, "call moneyness": "Deep ITM", "put mon
    {"strike": 95.0, "K/S0": 0.95, "call moneyness": "ITM", "put moneyne
    {"strike": 100.0, "K/S0": 1.00, "call moneyness": "ATM", "put moneyne
    {"strike": 105.0, "K/S0": 1.05, "call moneyness": "OTM", "put moneyne
    {"strike": 110.0, "K/S0": 1.10, "call moneyness": "Deep OTM", "put mo
]

strikes = [row["strike"] for row in strike_rows]
strike_table = pd.DataFrame(strike_rows)
display(strike_table)
```

	strike	K/S0	call moneyness	put moneyness
0	90.000000	0.900000	Deep ITM	Deep OTM
1	95.000000	0.950000	ITM	OTM
2	100.000000	1.000000	ATM	ATM
3	105.000000	1.050000	OTM	ITM
4	110.000000	1.100000	Deep OTM	Deep ITM

Q15 — European call prices using a trinomial tree

```
In [14]: q15_rows = []
for row in strike_rows:
    result = trinomial_tree_price(S0, row["strike"], r, sigma, T, TRINOMI
    q15_rows.append({
        "question": "Q15",
        "style": "European",
        "option": "call",
        "strike": row["strike"],
        "K/S0": row["K/S0"],
        "moneyness": row["call moneyness"],
        "steps": TRINOMIAL_STEPS,
        "price": cents(result["price"]),
    })

q15_answer = pd.DataFrame(q15_rows)
display(q15_answer)
```

	question	style	option	strike	K/S0	moneyness	steps	price
0	Q15	European	call	90.000000	0.900000	Deep ITM	1000	11.670000
1	Q15	European	call	95.000000	0.950000	ITM	1000	7.710000
2	Q15	European	call	100.000000	1.000000	ATM	1000	4.610000
3	Q15	European	call	105.000000	1.050000	OTM	1000	2.480000
4	Q15	European	call	110.000000	1.100000	Deep OTM	1000	1.190000

Q15 answer. European call prices decrease as the strike increases. This pattern is economically sensible because a higher strike makes it less likely that the call finishes in the money.

Q16 — European put prices using a trinomial tree

```
In [15]: q16_rows = []
for row in strike_rows:
    result = trinomial_tree_price(S0, row["strike"], r, sigma, T, TRINOMI
    q16_rows.append({
        "question": "Q16",
        "style": "European",
        "option": "put",
```

```

        "strike": row["strike"],
        "K/S0": row["K/S0"],
        "moneyness": row["put moneyness"],
        "steps": TRINOMIAL_STEPS,
        "price": cents(result["price"]),
    })

q16_answer = pd.DataFrame(q16_rows)
display(q16_answer)

```

	question	style	option	strike	K/S0	moneyness	steps	price
0	Q16	European	put	90.000000	0.900000	Deep OTM	1000	0.550000
1	Q16	European	put	95.000000	0.950000	OTM	1000	1.530000
2	Q16	European	put	100.000000	1.000000	ATM	1000	3.370000
3	Q16	European	put	105.000000	1.050000	ITM	1000	6.170000
4	Q16	European	put	110.000000	1.100000	Deep ITM	1000	9.820000

Q16 answer. European put prices increase as the strike increases. A higher strike gives the put holder the right to sell at a higher price, so the put becomes more valuable.

Q17 — American call prices using a trinomial tree

```

In [16]: q17_rows = []
for row in strike_rows:
    result = trinomial_tree_price(S0, row["strike"], r, sigma, T, TRINOMI
    q17_rows.append({
        "question": "Q17",
        "style": "American",
        "option": "call",
        "strike": row["strike"],
        "K/S0": row["K/S0"],
        "moneyness": row["call moneyness"],
        "steps": TRINOMIAL_STEPS,
        "price": cents(result["price"]),
    })

q17_answer = pd.DataFrame(q17_rows)
display(q17_answer)

```

	question	style	option	strike	K/S0	moneyness	steps	price
0	Q17	American	call	90.000000	0.900000	Deep ITM	1000	11.670000
1	Q17	American	call	95.000000	0.950000	ITM	1000	7.710000
2	Q17	American	call	100.000000	1.000000	ATM	1000	4.610000
3	Q17	American	call	105.000000	1.050000	OTM	1000	2.480000
4	Q17	American	call	110.000000	1.100000	Deep OTM	1000	1.190000

Q17 answer. American call prices match European call prices up to rounding. This is expected for non-dividend-paying stocks because early exercise of a call sacrifices time value without providing a dividend benefit.

Q18 — American put prices using a trinomial tree

```
In [17]: q18_rows = []
for row in strike_rows:
    result = trinomial_tree_price(S0, row["strike"], r, sigma, T, TRINOMI
    q18_rows.append({
        "question": "Q18",
        "style": "American",
        "option": "put",
        "strike": row["strike"],
        "K/S0": row["K/S0"],
        "moneyness": row["put moneyness"],
        "steps": TRINOMIAL_STEPS,
        "price": cents(result["price"]),
    })

q18_answer = pd.DataFrame(q18_rows)
display(q18_answer)
```

	question	style	option	strike	K/S0	moneyness	steps	price
0	Q18	American	put	90.000000	0.900000	Deep OTM	1000	0.560000
1	Q18	American	put	95.000000	0.950000	OTM	1000	1.570000
2	Q18	American	put	100.000000	1.000000	ATM	1000	3.480000
3	Q18	American	put	105.000000	1.050000	ITM	1000	6.420000
4	Q18	American	put	110.000000	1.100000	Deep ITM	1000	10.330000

Q18 answer. American put prices are at least as large as European put prices. The early-exercise premium is larger for higher strikes because those puts are deeper in the money and can benefit more from exercising before maturity.

Consolidated Q5-Q18 answer tables

```
In [18]: binomial_summary = pd.concat(
    [
        q5_answer,
        q8_answer,
    ],
    ignore_index=True,
)

delta_summary = pd.concat(
    [
        q6_answer,
        q9_answer,
    ],
    ignore_index=True,
```

```

)

vol_summary = pd.concat(
    [
        q7_answer,
        q10_answer,
    ],
    ignore_index=True,
)

trinomial_summary = pd.concat(
    [
        q15_answer,
        q16_answer,
        q17_answer,
        q18_answer,
    ],
    ignore_index=True,
)

display(binomial_summary)
display(delta_summary)
display(vol_summary)
display(trinomial_summary)

```

	question	style	option	strike	steps	price
0	Q5	European	call	100.000000	1000	4.610000
1	Q5	European	put	100.000000	1000	3.370000
2	Q8	American	call	100.000000	1000	4.610000
3	Q8	American	put	100.000000	1000	3.480000

	question	style	option	delta
0	Q6	European	call	0.569400
1	Q6	European	put	-0.430600
2	Q9	American	call	0.569400
3	Q9	American	put	-0.449500

	question	style	option	price at 20% vol	price at 25% vol	price change for +5 vol points	approx change per +1 vol point
0	Q7	European	call	4.610000	5.600000	0.980000	0.200000
1	Q7	European	put	3.370000	4.350000	0.980000	0.200000
2	Q10	American	call	4.610000	5.600000	0.980000	0.200000
3	Q10	American	put	3.480000	4.460000	0.980000	0.200000

	question	style	option	strike	K/S0	moneyness	steps	price
0	Q15	European	call	90.000000	0.900000	Deep ITM	1000	11.670000
1	Q15	European	call	95.000000	0.950000	ITM	1000	7.710000
2	Q15	European	call	100.000000	1.000000	ATM	1000	4.610000
3	Q15	European	call	105.000000	1.050000	OTM	1000	2.480000
4	Q15	European	call	110.000000	1.100000	Deep OTM	1000	1.190000
5	Q16	European	put	90.000000	0.900000	Deep OTM	1000	0.550000
6	Q16	European	put	95.000000	0.950000	OTM	1000	1.530000
7	Q16	European	put	100.000000	1.000000	ATM	1000	3.370000
8	Q16	European	put	105.000000	1.050000	ITM	1000	6.170000
9	Q16	European	put	110.000000	1.100000	Deep ITM	1000	9.820000
10	Q17	American	call	90.000000	0.900000	Deep ITM	1000	11.670000
11	Q17	American	call	95.000000	0.950000	ITM	1000	7.710000
12	Q17	American	call	100.000000	1.000000	ATM	1000	4.610000
13	Q17	American	call	105.000000	1.050000	OTM	1000	2.480000
14	Q17	American	call	110.000000	1.100000	Deep OTM	1000	1.190000
15	Q18	American	put	90.000000	0.900000	Deep OTM	1000	0.560000
16	Q18	American	put	95.000000	0.950000	OTM	1000	1.570000
17	Q18	American	put	100.000000	1.000000	ATM	1000	3.480000
18	Q18	American	put	105.000000	1.050000	ITM	1000	6.420000
19	Q18	American	put	110.000000	1.100000	Deep ITM	1000	10.330000

Notes for the written report

- European put-call parity holds within sensible rounding for the European binomial prices.
- American options should be checked using no-arbitrage bounds rather than strict European parity equality.
- American calls match European calls in this no-dividend setup.
- American puts are more valuable than European puts because early exercise can add value.
- Increasing volatility increases both call and put values because both options benefit from a wider terminal stock-price distribution.